

dff9-W4111-Fall-2025-002-

HW2.ipynb

Introduction

Homework Overview

There is one version of the homework that both the programming and non-programming tracks complete.

There are 4 categories of question:

1. Short answer questions testing general knowledge and the ability to reason about the course material.
2. Entity-Relationship modeling questions testing the ability to define an ER model from a text/written description of a problem and to translate an ER model into SQL.
3. Relational algebra questions testing the ability to use the relational algebra.
4. SQL questing testing the ability to use SQL to produce data results from descriptions of the desired output.

Submission Instructions

The homework is due at 11:59 PM on Sunday, 08-March-2026.

Please see [Ed discussion](#) for detailed submission instructions and for clarifications and answering questions.

Part 1 — Setup

iPython-SQL

```
In [41]: %load_ext sql
```

The sql extension is already loaded. To reload it, use:
%reload_ext sql

Set the proper root user ID and password for MySQL in the python cell below.

```
In [42]: mysql_root_user = 'root'
mysql_root_password = 'david-database'

mysql_url = f"mysql+pymysql://{mysql_root_user}:{mysql_root_password}@localhost"
```

```
In [43]: mysql_url
```

Out[43]: 'mysql+pymysql://root:david-database@localhost'

```
In [44]: %sql $mysql_url
```

```
In [45]: %config SqlMagic.style = '_DEPRECATED_DEFAULT'
```

```
In [46]: %sql select * from db_book.student where dept_name="Comp. Sci."
```

* mysql+pymysql://root:***@localhost
4 rows affected.

Out[46]:

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
54321	Williams	Comp. Sci.	54
76543	Brown	Comp. Sci.	58

SQL Alchemy

```
In [47]: from sqlalchemy import create_engine, text
```

```
In [48]: engine = create_engine(mysql_url)
```

```
In [49]: # Raw SQL query
sql = text("SELECT * FROM db_book.student WHERE dept_name = :dept")

# Execute query
with engine.connect() as connection:
    result = connection.execute(sql, {"dept": "Comp. Sci."})

    for row in result:
        print(row)

('00128', 'Zhang', 'Comp. Sci.', Decimal('102'))
('12345', 'Shankar', 'Comp. Sci.', Decimal('32'))
('54321', 'Williams', 'Comp. Sci.', Decimal('54'))
('76543', 'Brown', 'Comp. Sci.', Decimal('58'))
```

Pandas

```
In [50]: import pandas

In [51]: result = pandas.read_sql(con=engine, sql="select * from db_book.student where dept_name='Comp. Sci.'")

In [52]: result
```

	ID	name	dept_name	tot_cred
0	00128	Zhang	Comp. Sci.	102.0
1	12345	Shankar	Comp. Sci.	32.0
2	54321	Williams	Comp. Sci.	54.0
3	76543	Brown	Comp. Sci.	58.0

PyMySQL

```
In [53]: import pymysql

In [54]: conn = pymysql.connect(
    host="localhost",
    user=mysql_root_user,
    password=mysql_root_password,
    cursorclass=pymysql.cursors.DictCursor,
    autocommit=True
)

In [55]: cursor = conn.cursor()

In [56]: result = cursor.execute("select * from db_book.student where dept_name='Comp. Sci.'")
result = cursor.fetchall()

In [57]: result
```

```
Out[57]: [{ 'ID': '00128',
  'name': 'Zhang',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('102')},
{ 'ID': '12345',
  'name': 'Shankar',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('32')},
{ 'ID': '54321',
  'name': 'Williams',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('54')},
{ 'ID': '76543',
  'name': 'Brown',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('58')}]
```

Written Questions

The written questions test general knowledge of the material presented in lectures and in the slides from the recommended textbook.

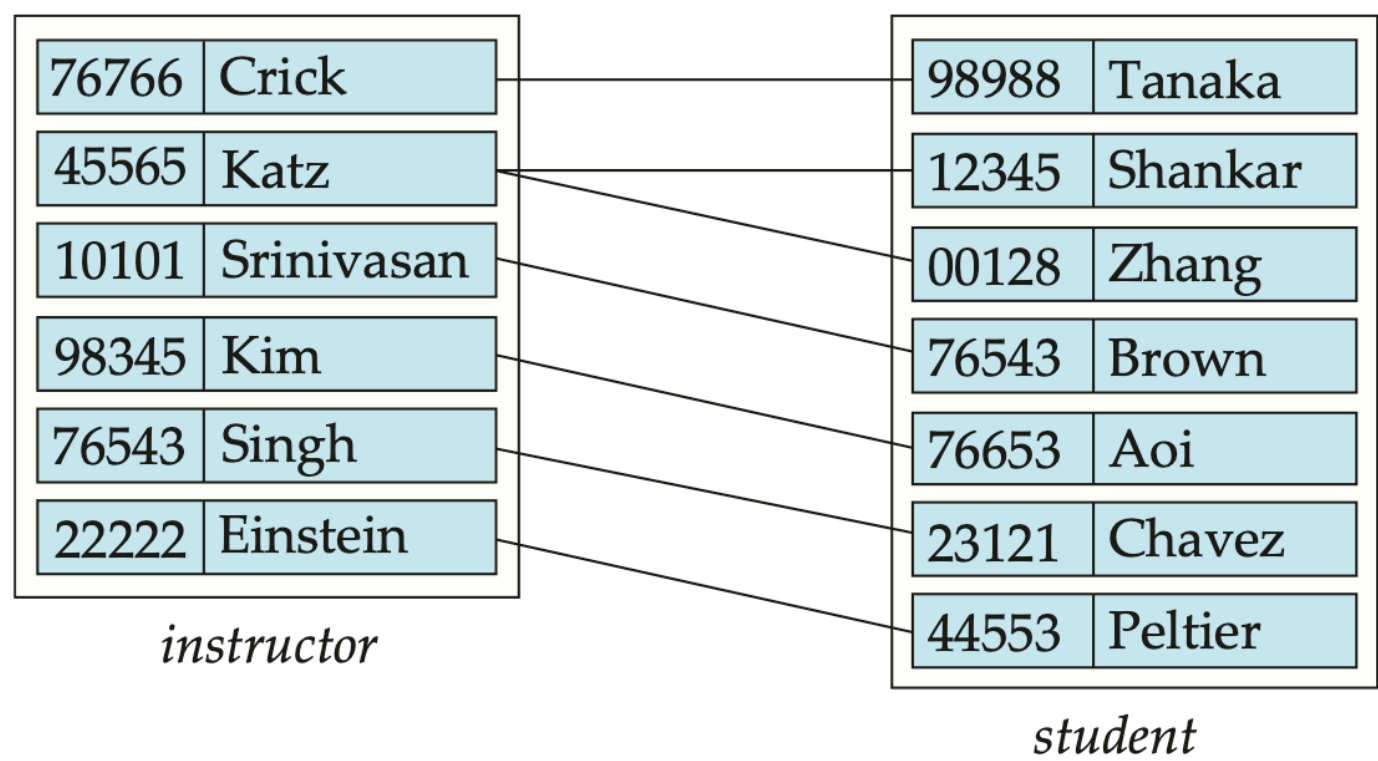
Your answers should be short and simple, and typically less than 5 sentences.

We will deduct points for answers that are rambling, too long, throwing out ideas and hoping something is correct, etc.

Q1

Question

Using the approach/notation from lecture 3, slides 5 and 6 (also labelled 6.5, 6.6), what is the *relationship set* for the following diagrams?



Answer

{(76766,98988),(45565,12345),(45565,00128),(10101,76543),(98345,76653),(76543,23121),(22222,44553)}

Q2

Question

What are the two primary reasons for using an *associative entity* to implement a relationship between two entity sets?

Answer

1. when the relationship is many-to-many, and
2. when the relationship has its own attributes/properties.

Q3

Question

Briefly explain the concept of a *weak entity*.

Answer

A weak entity is an entity set that cannot be uniquely identified by its own attributes alone. Its existence depends on another entity, called the identifying entity, and it is uniquely identified using the identifying entity's key together with its own discriminator attributes.

Q4

Question

What are the 4 types of integrity constraint on a single relation/table in SQL?

Answer

NOT NULL
PRIMARY KEY
UNIQUE
CHECK

Q5

Question

What is the key difference between a *unique* constraint and a *primary key* constraint?

Answer

PRIMARY KEY uniquely identifies each row and cannot contain NULL values, while a UNIQUE constraint also enforces uniqueness but is allowed to contain NULL values. UNIQUE is described as defining a candidate key.

Q6

Question

Briefly explain the concepts of *disjoint/overlapping* and *total/partial* with respect to inheritance.

Answer

Disjoint means an entity in the higher-level entity set can belong to only one subclass. Overlapping means the same entity can belong to more than one subclass.

Total means every entity in the higher-level entity set must belong to at least one subclass. Partial means some entities in the higher-level entity set may belong to no subclass.

Q7

Question

Briefly explain the following concepts:

1. Natural join
2. Theta join
3. Inner join
4. Outer join

Answer

Natural join joins two relations by matching columns with the same name and equal values.

Theta join joins two relations using any predicate/condition.

Inner join returns only rows that satisfy the join condition in both relations.

Outer join returns matching rows plus non-matching rows from one or both relations, using NULL for missing values.

Q8

Question

Assume that a database implementation does not support `FULL OUTER JOIN`. How is it possible to implement an equivalent query using other operations?

Answer

If FULL OUTER JOIN is not supported, you can get the same result by taking a LEFT OUTER JOIN and a RIGHT OUTER JOIN on the same condition, and then combining the two results with UNION. This returns all matching rows plus all non-matching rows from both sides.

Q9

Question

What are the 3 types of Join conditions?

Answer

The three types of join conditions discussed in the slides are natural join, equijoin, and theta join. A natural join matches same-named columns automatically, an equijoin uses explicit equality conditions, and a theta join uses an arbitrary predicate.

Q10

Question

Briefly explain **Codd's 4th Rule: Active Online Catalog:** *The structure description of the entire database must be stored in an online catalog, known as data dictionary, which can be accessed by authorized users. Users can use the same query language to access the catalog which they use to access the database itself.*

What is the name of the schema/database for the catalog in MySQL?

Answer

Codd’s 4th Rule says that the database’s structure and metadata should be stored in an online catalog, or data dictionary, that authorized users can query using the same relational/query language used for normal data. In other words, metadata should be accessible just like regular database data.

The MySQL schema/database for this catalog is information_schema

Entity Relationship Modelling

ER1

Question

Consider the following description of a datamodel.

A company sells products to customers. The operational database contains the following tables.

Your task is to draw a **logical ER diagram using Crow’s Foot notation** based solely on the description below.

From the description, you must determine:

- Primary keys
- Composite keys
- Foreign keys
- Cardinalities
- Optional vs. mandatory participation
- Whether any entity is associative

Customer

Each **Customer** represents a company that purchases products.

Attributes

- customerNumber — unique identifier for a customer
- customerName
- contactLastName
- contactFirstName
- phone
- addressLine1
- addressLine2
- city
- state
- postalCode
- country
- creditLimit

Business Rules

- Each customer has a unique **customerNumber** .
- A customer may place zero, one, or many orders.
- A customer may make zero, one, or many payments.

Orders

Each **Order** represents a purchase transaction placed by a customer.

Attributes

- orderNumber — unique identifier for an order
- orderDate
- requiredDate
- shippedDate
- status

- comments

Business Rules

- Each order has a unique `orderNumber` .
- Each order is placed by exactly one customer.
- An order must contain at least one product.
- An order may contain many products.

OrderDetails

The **OrderDetails** table represents the individual line items of an order.

Attributes

- orderNumber
- productCode
- quantityOrdered
- priceEach
- orderLineNumber

Business Rules

- Each row represents one product appearing on one order.
- The combination of `orderNumber` and `productCode` uniquely identifies a line item.
- Each line item belongs to exactly one order.
- Each line item references exactly one product.

Products

Each **Product** represents an item available for sale.

Attributes

- productCode — unique identifier for a product
- productName
- productLine
- productScale
- productVendor
- productDescription
- quantityInStock
- buyPrice
- MSRP

Business Rules

- Each product has a unique `productCode` .
- A product may appear in zero, one, or many order line items.

Payments

Each **Payment** represents a payment made by a customer.

Attributes

- customerNumber
- checkNumber
- paymentDate
- amount

Business Rules

- Each payment is made by exactly one customer.
- A customer may make multiple payments.
- The combination of `customerNumber` and `checkNumber` uniquely identifies a payment.

Deliverable

Draw a **Crow’s Foot ER diagram** that includes:

- All entities
- All attributes
- Primary keys (clearly marked)
- Composite keys (clearly marked)

- All relationships and foreign keys
- Cardinalities
- Optional vs. mandatory participation

Do not include implementation details such as data types.

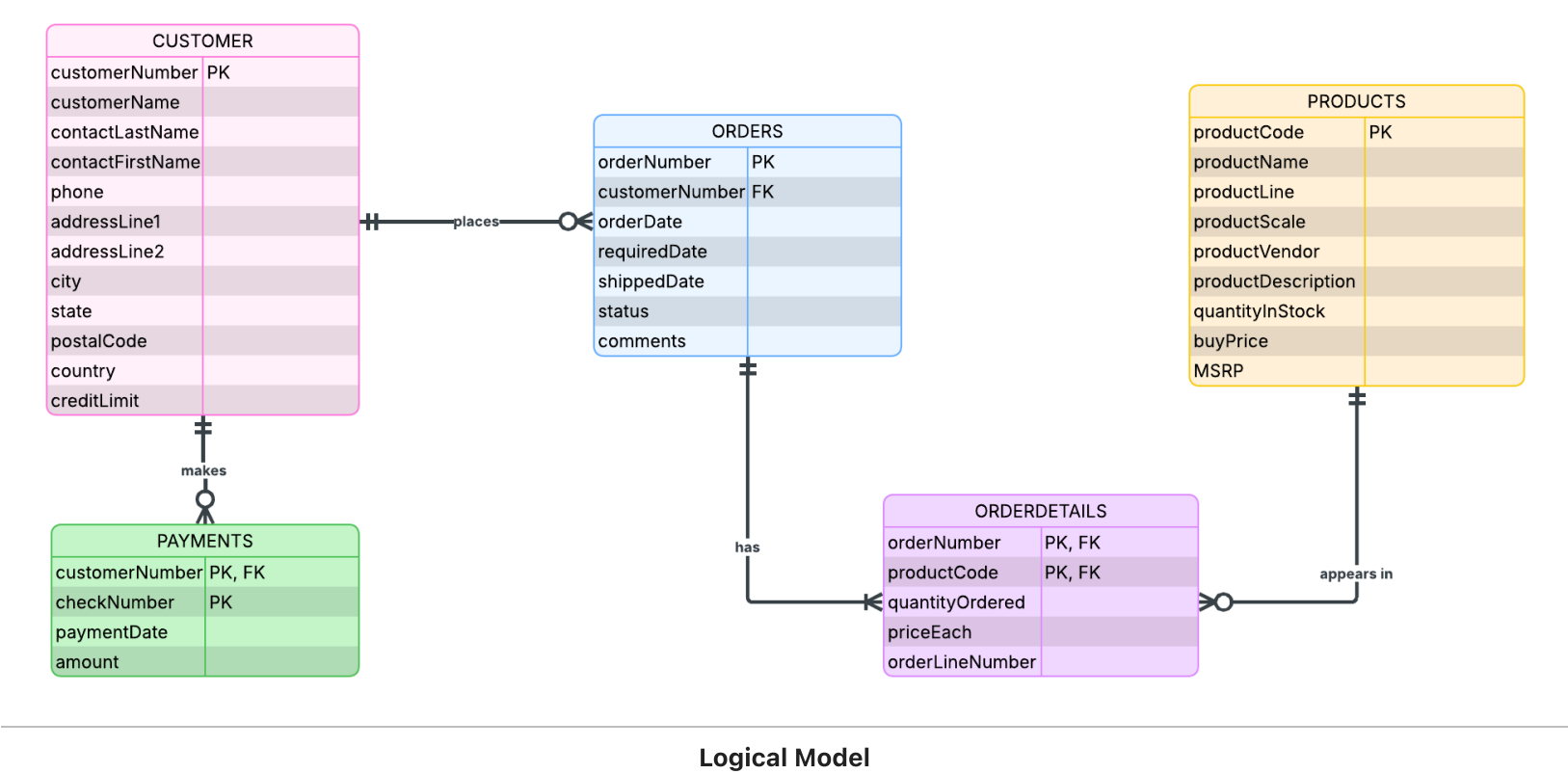
Using Crow's Foot Notation and Lucidchart or an equivalent tool, draw a **logical datamodel** for the scenario. Replace the image below with a screen capture of your diagram.

You may have to define additional attributes or entity types to implement your solution.

You may add notes to the diagrams to document any assumptions, explain concepts your could not fully express in Crow's Foot Notation, etc.

NOTE: The diagram below is a conceptual model. The question is asking you to define a logical model.

Answer



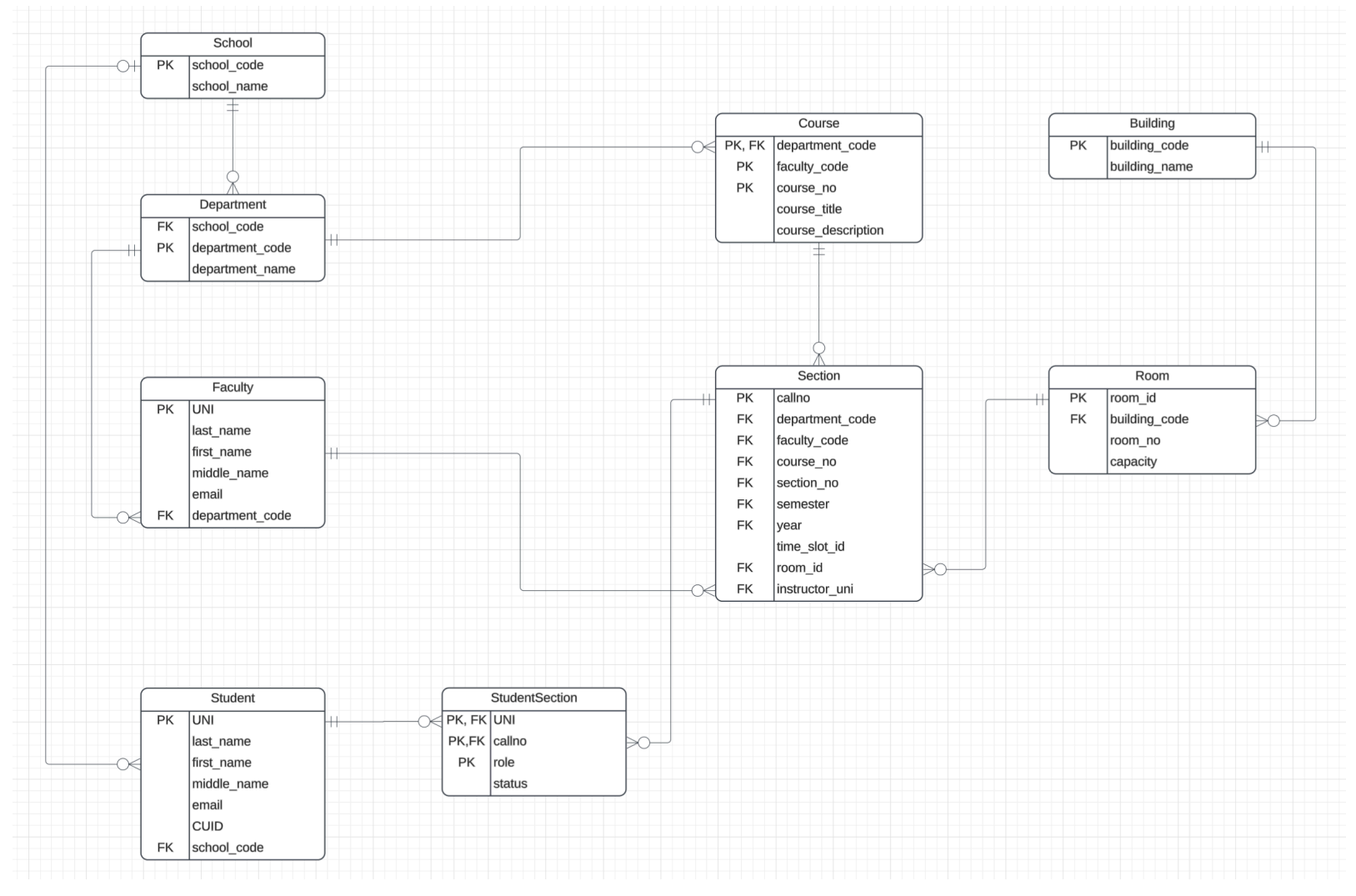
ER2

Question

You are to translate/write DDL to create an SQL database implementing the following diagram. Unless otherwise specified, you can assume that all columns are `VARCHAR(64)`.

The model has the additional constraints:

1. All `email` columns are `unique` within the table.
2. `School.school_code` is always exactly 4 characters.
3. `Department.department_code` is always exactly 4 characters.
4. `CUID` is `unique` and `not null`.
5. `Course.faculty_code` and `Section.course_code` are a single characters and the allowed values are "C", "B", "E" and "W".
6. `course_no` is always exactly 4 characters and only the digits "0", "1", ..., "9" are allowed characters.
7. The allowed values for `Section.semester` are 1, 2 and 3.
8. `StudentSection.role` can have the values `Enrolled`, `Waitlist`, `TA`.



Logical Model

Answer

```

In [58]: %%sql

drop database if exists w4111_hw2;
create database w4111_hw2;
  
```

* mysql+pymysql://root:***@localhost
 9 rows affected.
 1 rows affected.

Out[58]: []

```

In [59]: %%sql

USE w4111_hw2;

CREATE TABLE School (
    school_code CHAR(4) NOT NULL,
    school_name VARCHAR(64) NOT NULL,
    CONSTRAINT pk_school PRIMARY KEY (school_code),
    CONSTRAINT chk_school_code_len CHECK (CHAR_LENGTH(school_code) = 4)
);

CREATE TABLE Department (
    school_code CHAR(4) NOT NULL,
    department_code CHAR(4) NOT NULL,
    department_name VARCHAR(64) NOT NULL,
    CONSTRAINT pk_department PRIMARY KEY (department_code),
    CONSTRAINT fk_department_school FOREIGN KEY (school_code) REFERENCES School(school_code),
    CONSTRAINT chk_department_code_len CHECK (CHAR_LENGTH(department_code) = 4)
);

CREATE TABLE Faculty (
    UNI VARCHAR(64) NOT NULL,
    last_name VARCHAR(64),
    first_name VARCHAR(64),
    middle_name VARCHAR(64),
    email VARCHAR(64),
    department_code CHAR(4) NOT NULL,
    CONSTRAINT pk_faculty PRIMARY KEY (UNI),
    CONSTRAINT uq_faculty_email UNIQUE (email),
    CONSTRAINT fk_faculty_department FOREIGN KEY (department_code) REFERENCES Department(department_code)
);

CREATE TABLE Student (
    UNI VARCHAR(64) NOT NULL,
    last_name VARCHAR(64),
    first_name VARCHAR(64),
    middle_name VARCHAR(64),
    email VARCHAR(64),
    CUID VARCHAR(64) NOT NULL,
    school_code CHAR(4) NOT NULL,
  
```



```

    CONSTRAINT pk_student PRIMARY KEY (UNI),
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT uq_student_cuid UNIQUE (CUID),
    CONSTRAINT fk_student_school
        FOREIGN KEY (school_code) REFERENCES School(school_code)
);

CREATE TABLE Building (
    building_code VARCHAR(64) NOT NULL,
    building_name VARCHAR(64) NOT NULL,
    CONSTRAINT pk_building PRIMARY KEY (building_code)
);

CREATE TABLE Room (
    room_id VARCHAR(64) NOT NULL,
    building_code VARCHAR(64) NOT NULL,
    room_no VARCHAR(64) NOT NULL,
    capacity INT,
    CONSTRAINT pk_room PRIMARY KEY (room_id),
    CONSTRAINT fk_room_building
        FOREIGN KEY (building_code) REFERENCES Building(building_code)
);

CREATE TABLE Course (
    department_code CHAR(4) NOT NULL,
    faculty_code CHAR(1) NOT NULL,
    course_no CHAR(4) NOT NULL,
    course_title VARCHAR(64) NOT NULL,
    course_description VARCHAR(64),
    CONSTRAINT pk_course PRIMARY KEY (department_code, faculty_code, course_no),
    CONSTRAINT fk_course_department
        FOREIGN KEY (department_code) REFERENCES Department(department_code),
    CONSTRAINT chk_course_faculty_code
        CHECK (faculty_code IN ('C', 'B', 'E', 'W')),
    CONSTRAINT chk_course_no_format
        CHECK (course_no REGEXP '^[0-9]{4}$')
);

CREATE TABLE Section (
    callno VARCHAR(64) NOT NULL,
    department_code CHAR(4) NOT NULL,
    faculty_code CHAR(1) NOT NULL,
    course_no CHAR(4) NOT NULL,
    section_no VARCHAR(64) NOT NULL,
    semester CHAR(1) NOT NULL,
    year VARCHAR(64) NOT NULL,
    time_slot_id VARCHAR(64),
    room_id VARCHAR(64),
    instructor_uni VARCHAR(64),
    CONSTRAINT pk_section PRIMARY KEY (callno),
    CONSTRAINT fk_section_course
        FOREIGN KEY (department_code, faculty_code, course_no)
        REFERENCES Course(department_code, faculty_code, course_no),
    CONSTRAINT fk_section_room
        FOREIGN KEY (room_id) REFERENCES Room(room_id),
    CONSTRAINT fk_section_faculty
        FOREIGN KEY (instructor_uni) REFERENCES Faculty(UNI),
    CONSTRAINT uq_section_natural
        UNIQUE (department_code, faculty_code, course_no, section_no, semester, year),
    CONSTRAINT chk_section_faculty_code
        CHECK (faculty_code IN ('C', 'B', 'E', 'W')),
    CONSTRAINT chk_section_course_no_format
        CHECK (course_no REGEXP '^[0-9]{4}$'),
    CONSTRAINT chk_section_semester
        CHECK (semester IN ('1', '2', '3'))
);

CREATE TABLE StudentSection (
    UNI VARCHAR(64) NOT NULL,
    callno VARCHAR(64) NOT NULL,
    role VARCHAR(64) NOT NULL,
    status VARCHAR(64),
    CONSTRAINT pk_student_section PRIMARY KEY (UNI, callno, role),
    CONSTRAINT fk_student_section_student
        FOREIGN KEY (UNI) REFERENCES Student(UNI),
    CONSTRAINT fk_student_section_section
        FOREIGN KEY (callno) REFERENCES Section(callno),
    CONSTRAINT chk_student_section_role
        CHECK (role IN ('Enrolled', 'Waitlist', 'TA'))
);

```

```
* mysql+pymysql://root:***@localhost
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
```

Out[59]: []

Relational Algebra Questions

Using the [RelaX relational calculator](#) and the sample data that comes with the recommended textbook for the following questions.

You answer will be a text copy/paste of your query and a screen capture/snip of the execution of the query.

R1

Question

Write an algebra expression that computes a relation of the form (ID, name) that includes the ID and name of instructors that do not advise any students. You will use the relations instructor and advisor. My result looks like

ID	instructor_name
12121	'Wu'
15151	'Mozart'
32343	'El Said'
33456	'Gold'
58583	'Califieri'
83821	'Brandt'

The answer area below contains an example of the result format but for a different query.

Answer

Relational Algebra Statement

```
π ID←instructor.ID, instructor_name←instructor.name (instructor)
−
π ID←instructor.ID, instructor_name←instructor.name
(
    instructor ⋈ instructor.ID=advisor.i_id advisor
)
```

Execution Screen Capture

execute selection

download

history

-

6 rows

π instructor.ID→ID, instructor.name→instructor_name

12 rows

π instructor.ID→ID, instructor.name→instructor_name

6 rows

instructor

12 rows

(⋈ instructor.ID = advisor.i_id)

9 rows

instructor

12 rows

advisor

9 rows

π instructor.ID→ID, instructor.name→instructor_name (instructor) - π instructor.ID→ID, instructor.name→instructor_name (instructor ⋈ instructor.ID = advisor.i_id advisor)

Execution time: 4 ms

ID	instructor_name
12121	'Wu'
15151	'Mozart'
32343	'El Said'
33456	'Gold'
58583	'Califieri'
83821	'Brandt'

R1 Answer

R2

Question

Write a relational algebra expression that produces a table of for the (ID, name, course_id, title, credits) that contains the ID and name of instructors and the course_id, title and credits for the courses they teach. Your result should only contain faculty in the 'Comp. Sci.' department. Your expression will use the relations instructor , teaches and course .

My answer looks like

ID	name	course_id	title	credits
10101	'Srinivasan'	'CS-101'	'Intro. to Computer Science'	4
10101	'Srinivasan'	'CS-315'	'Robotics'	3
10101	'Srinivasan'	'CS-347'	'Database System Concepts'	3
45565	'Katz'	'CS-101'	'Intro. to Computer Science'	4
45565	'Katz'	'CS-319'	'Image Processing'	3
83821	'Brandt'	'CS-190'	'Game Design'	4
83821	'Brandt'	'CS-319'	'Image Processing'	3

The answer area below contains an example of the result format but for a different query.

Answer

Relational Algebra Statement

```

π ID←instructor.ID, name←instructor.name, course_id←teaches.course_id,
title←course.title, credits←course.credits
(
  ((σ instructor.dept_name='Comp. Sci.' instructor) ⋈ instructor.ID=teaches.ID teaches)
  ⋈ teaches.course_id=course.course_id course
)
```

Execution Screen Capture

2010.

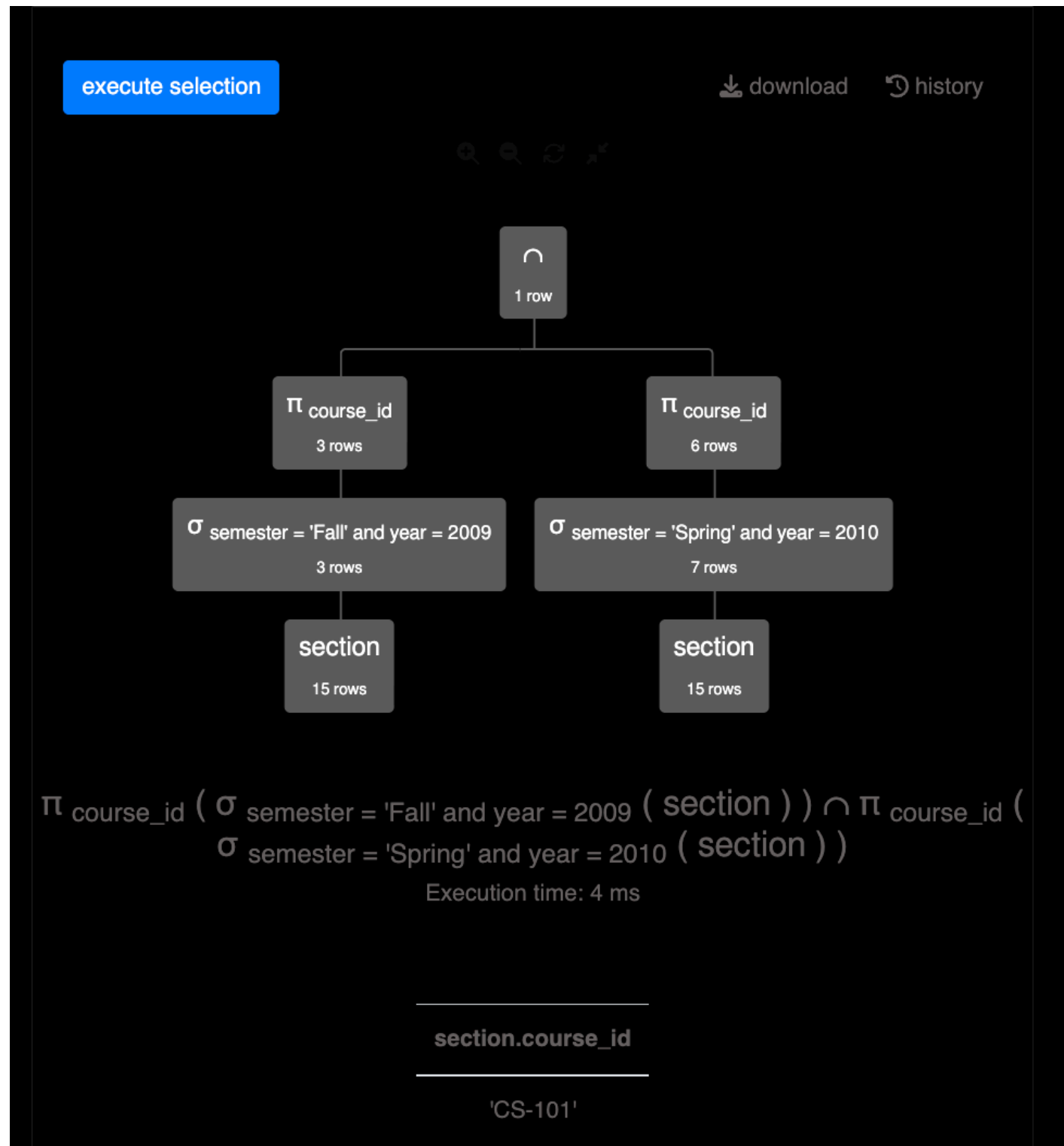
The answer area below contains an example of the result format but for a different query.

Answer

Relational Algebra Statement

$\pi_{course_id}(\sigma_{semester='Fall' \wedge year=2009}(section))$
 $\cap$
 $\pi_{course_id}(\sigma_{semester='Spring' \wedge year=2010}(section))$

Execution Screen Capture



R3 Answer

SQL

S1

Question

Write and execute the SQL query that is equivalent to the relational algebra expression for question R1.

Answer

```
In [61]: %%sql
USE db_book;

SELECT i.ID, i.name AS instructor_name
FROM instructor AS i
WHERE NOT EXISTS (
    SELECT 1
    FROM advisor AS a
```



```
WHERE a.i_id = i.ID
);
```

* mysql+pymysql://root:***@localhost
0 rows affected.
6 rows affected.

Out[61]:

ID	instructor_name
12121	Wu
15151	Mozart
32343	El Said
33456	Gold
58583	Califieri
83821	Brandt

S2

Question

Write and execute the SQL query that is equivalent to the relational algebra expression for question R2.

Answer

```
In [67]: %%sql

USE db_book;

SELECT DISTINCT i.ID, i.name, c.course_id, c.title, c.credits
FROM instructor AS i
JOIN teaches AS t
    ON i.ID = t.ID
JOIN course AS c
    ON t.course_id = c.course_id
WHERE i.dept_name = 'Comp. Sci.'
ORDER BY i.ID, c.course_id;

* mysql+pymysql://root:***@localhost
0 rows affected.
7 rows affected.
```

Out[67]:

ID	name	course_id	title	credits
10101	Srinivasan	CS-101	Intro. to Computer Science	4
10101	Srinivasan	CS-315	Robotics	3
10101	Srinivasan	CS-347	Database System Concepts	3
45565	Katz	CS-101	Intro. to Computer Science	4
45565	Katz	CS-319	Image Processing	3
83821	Brandt	CS-190	Game Design	4
83821	Brandt	CS-319	Image Processing	3

S3

Question

Write and execute the SQL query that is equivalent to the relational algebra expression for question R3.

NOTE:

- 1. You need to change the semesters to Fall 2017 and Spring 2018 because the data is different.
- 2. MySQL does not support `INTERSECT` . So, you will have to choose a different approach to performing the "intersection."

Answer

```
In [64]: %%sql

SELECT f.course_id
FROM (
    SELECT DISTINCT course_id
    FROM section
    WHERE semester = 'Fall' AND year = 2017
) AS f
JOIN (
    SELECT DISTINCT course_id
    FROM section
    WHERE semester = 'Spring' AND year = 2018
```

```
) AS s
ON f.course_id = s.course_id;
```

* mysql+pymysql://root:***@localhost
1 rows affected.

Out[64]:

course_id
CS-101

S4

Question

Without using any kind of JOIN, for the sample dataset, write a SQL query that produces a result of the form (student_name, instructor_name) that contains the name of a student and the student's advisor. Your query will use the tables student, advisor and instructor . Your query should only contain results for instructors in the dept_name "Comp. Sci." .

For reference my query produces

	student_name	instructor_name
1	Shankar	Srinivasan
2	Zhang	Katz
3	Brown	Katz

Answer

```
In [68]: %%sql

USE db_book;

SELECT
(
    SELECT s.name
    FROM student AS s
    WHERE s.ID = a.s_id
) AS student_name,
(
    SELECT i.name
    FROM instructor AS i
    WHERE i.ID = a.i_id
) AS instructor_name
FROM advisor AS a
WHERE a.i_id IN (
    SELECT ID
    FROM instructor
    WHERE dept_name = 'Comp. Sci.'
);
```

* mysql+pymysql://root:***@localhost
0 rows affected.
3 rows affected.

Out[68]:

student_name	instructor_name
Shankar	Srinivasan
Zhang	Katz
Brown	Katz

S5

Question

Using the "online metadata catalog" for MySQL, write a query that produces a table of the form (table_schema, table_name, column_name, ordinal_position, column_default, is_nullable, data_type) for columns in the table db_book.student . For reference, my answer looks like

	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE
1	db_book	student	ID		<null>	NO	varchar
2	db_book	student	name		<null>	NO	varchar
3	db_book	student	dept_name		<null>	YES	varchar
4	db_book	student	tot_cred		<null>	YES	decimal

Answer

```
In [66]: %%sql

SELECT
```



```
table_schema,
table_name,
column_name,
ordinal_position,
column_default,
is_nullable,
data_type
FROM information_schema.columns
WHERE table_schema = 'db_book'
AND table_name = 'student'
ORDER BY ordinal_position;
```

* mysql+pymysql://root:***@localhost
4 rows affected.

Out[66]:

TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE
db_book	student	ID	1	None	NO	varchar
db_book	student	name	2	None	NO	varchar
db_book	student	dept_name	3	None	YES	varchar
db_book	student	tot_cred	4	None	YES	decimal

Create PDF

1. Use the `File -> Save and Export Notebook as -> HTML` option to save your notebook as an HTML file.
2. Copy the created HTML file to the same directory as the notebook and images you created.
3. Open the HTML file in a browser.
4. Using the *browser's* `File -> Print` option print the HTML file to a PDF. This is your submission format.
5. Follow the submission instructions on Ed Discussion to submit your file to GradeScope.

In []: